Sprint Review and Retrospective: SNHU Travel

Brandon Witbrodt
Southern New Hampshire University

CS 250: Software Development Lifecycle

The SNHU Travel project gave me an opportunity to examine the software development life cycle (SDLC) from several positions on a Scrum team. ChadaTech wanted to develop a vacation booking system that would highlight popular travel packages and help SNHU Travel reach more customers. Our team charter established a five-week timeline and identified Christy as Product Owner, Ron as Scrum Master, Nicky as Developer, and Brian as Tester. I examined these perspectives through simulated assignments rather than a completed production release. This review evaluates what I learned about Scrum.

The Product Owner's responsibility became especially clear when I worked with the travel customer feedback. Interview and focus-group comments included requests for personalized destination recommendations, budget limits, special deals, and vacation categories such as cruises or eco-travel. As Product Owner, I had to think beyond a general request for a better website and consider which features would deliver the most customer value. The Developer role required a different perspective: understanding the priority and acceptance criteria well enough to build the correct feature. As Tester, I looked for specific, observable outcomes rather than assuming that a broadly worded story was testable. The Scrum Master facilitated events, identified impediments, and protected the Sprint Goal. Each role depended on the others.

User stories provided a practical connection between customer needs and development tasks. For example, customers wanted to compare vacations using price and other details instead of browsing options without a clear way to evaluate them. That request could become a story with defined comparison fields and acceptance criteria. My tester email to Christy showed why refinement matters: I asked which columns to display, whether users could set a price range, and

how much detail each description should contain. I also asked whether the “Top 10” destinations should appear together or as a slideshow and whether the proposed mobile application was too large to treat as one story. These questions exposed decisions that needed to be made before tests could be completed. Sprint Planning would select appropriately sized, prioritized stories; development and testing would then use the same acceptance criteria and Definition of Done to determine when work was complete. Our coursework analyzed this process rather than producing finished software.

The Developer Journal asked me to respond to a change in the development plan. Instead of continuing with the original assumptions, I proposed contacting the Product Owner for revised priorities, affected stories, and acceptance criteria. I also requested updated test scenarios and regression risks from the Tester. My sample message asked both roles to record answers in the shared backlog before the next development session. This approach protects the Sprint Goal while allowing future work to change based on new information. Under a more rigid waterfall sequence, discovering these issues after requirements approval could require formal rework across later phases. Short feedback cycles make the impact of changes easier to discuss.

Two communication samples stand out to me. In the tester email, I asked targeted questions about rankings, sorting, price filters, and the scope of the mobile feature. Direct answers could become measurable test conditions. In the developer email, I requested confirmation of revised priorities and testing expectations from both the Product Owner and Tester. That message encouraged a shared decision instead of separate assumptions. The team charter reinforced respectful communication, early reporting of blockers, and participation in Scrum events. A visible Product Backlog would preserve requirements and priorities, while a Sprint Backlog would show the team’s current plan. Sprint Planning would establish the goal;

the Daily Scrum would inspect progress and surface impediments; the Sprint Review would gather stakeholder feedback; and the Retrospective would turn lessons into process improvements. Backlog refinement, although not a formal Scrum event, would prepare stories for the next planning discussion.

Scrum's strongest advantage for SNHU Travel was its ability to incorporate feedback before too much effort was committed to the wrong feature. The interviews showed that users did not all want the same travel experience, and the tester questions revealed that even simple-looking requirements contained unresolved details. Scrum also promoted accountability through visible work and frequent coordination. Its disadvantages included meeting overhead, shifting priorities, and the difficulty of forecasting a five-week project. I believe Scrum was the better fit for this project because customer preferences and feature details were expected to evolve. A waterfall approach could be appropriate when requirements are stable and approvals are fixed, but SNHU Travel benefited more from early clarification and incremental feedback. My main retrospective improvement would be to require a brief refinement conversation and testable acceptance criteria before bringing a story into Sprint Planning. That would reduce confusion for developers and testers without giving up Agile flexibility.